Architecture des Réseaux Informatiques : Communication inter-processus par sockets

Notion de Socket

domaine, protocole, adressage

Modèles de communication

DATAGRAM & STREAM

Primitives de gestion des sockets

socket, bind, accept, listen, connect, …

Primitives de manipulation de l'adressage IP

résolution de noms, port de service, …

Communication inter-processus ou IPC

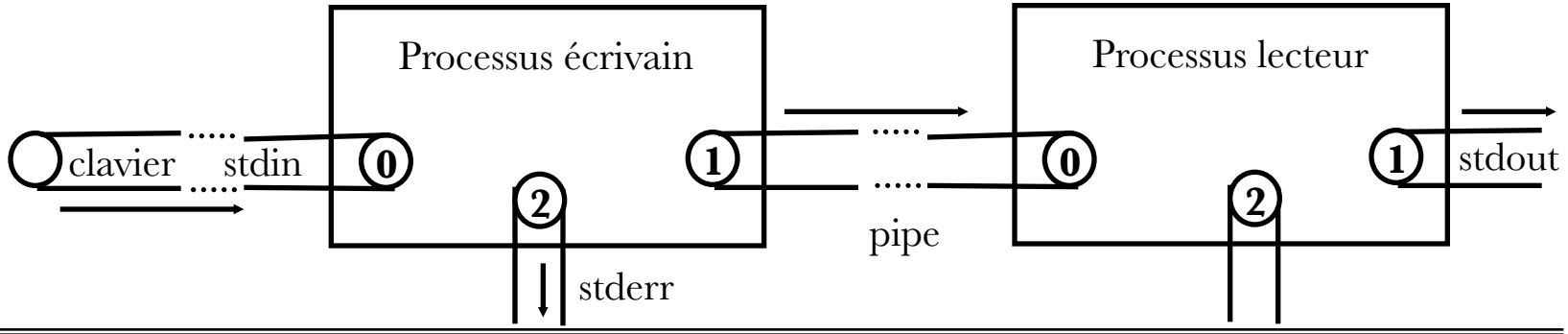
La communication entre deux processus se fait par l'utilisation d'un canal logique (rappel : tout processus possède par défaut trois canaux logiques : *STDIN*, *STDOUT*, *STDERR*).

Deux modes de communication sont possibles :

- ➢ connecté (téléphone) : un canal de bout en bout (attaché)
- ➢ non-connecté (courrier) : un canal non attaché

On distingue trois types d'IPC :

- ➢ **fichier** : entre deux processus distincts en mode non-connecté
- ➢ **pipe, pairsocket** : entre un processus **père** et un processus **fils**
- ➢ **socket** : entre deux processus **distincts.** Ce mécanisme prend tout son sens dans une application réseau (Client/Serveur).



Notion de socket

Les pipes et les pairsockets offrent une communication entre un processus père et un processus fils : nécessité d'avoir un ancêtre commun.

Deux processus créés séparément (éventuellement sur deux machines différentes : souvent le cas) qui veulent communiquer ensemble doivent utiliser le mécanisme de socket.

La notion de socket est introduite par UNIX BSD pour banaliser la communication dans le monde Internet.

- ➢Fournir un modèle **standard** de communication entre deux processus quelconques.
- ➢La communication se fait en **full-duplex**.
- ➢Une socket peut être assimilée à un fichier réseau → nécessité d'un mécanisme d'adressage universel.

Adressage & Domaine des sockets

L'initiative de communication est initiée par un processus. Par conséquent, le destinataire est connu à l'avance.

Une socket utilise un nom qui doit être translaté en adresse correspondante avant l'établissement de la communication.

La nature des sockets utilisées dépend du domaine d'application :

- AF_UNIX,
- AF_INET,
- AF_ROUTE,

Chaque domaine nécessite un système d'adressage pour désigner le destinataire :

- un chemin de fichier de type socket pour AF_UNIX,
- une adresse IP (hôte) et un numéro de port (service applicatif)

Protocole & mode de communication d'une socket

Le protocole définit les formats de données, les règles et les conventions régulant la communication.

Le protocole permet d'établir une connexion vers le destinataire et d'acheminer les données entre sockets.

Deux modes de communication sont possibles :

- **SOCK_DGRAM : DATAGRAM ou mode non connecté (courrier).**
- **SOCK_STREAM : STREAM ou mode connecté (Téléphone).**
- **SOCK_RAW : administrateur (couche 3 : IP, ICMP, …)**

Pour chaque mode de communication (STREAM, DATAGRAM) dans un domaine donné (AF_UNIX, AF_INET, …), il n'existe généralement qu'un seul protocole de communication.

Rappel : La création d'une socket nécessite la spécification du *domaine*, du *mode* et du *protocole*.

Création d'une socket

La création d'une socket se fait par l' appel système :

```
#include <sys/socket.h>
sock = socket(int domain, int mode, int protocol);
/*sock=numéro du canal logique attribué*/
```

La donnée *domain* indique la famille qui sera utilisée : *AF_UNIX*, *AF_INET*, ...

La donnée *mode* indique le mode d'échange qui sera utilisé : *SOCK_DGRAM*, *SOCK_STREAM*, *SOCK_RAW*.

Le *protocole* est implicite en fonction du domaine et mode qui seront choisis :

- la valeur 0 permet d'utiliser les protocoles par défaut
- protocole *UDP* pour AF_UNIX et AF_INET en mode SOCK_DGRAM
- protocole *TCP* pour AF_INET en SOCK_STREAM

En cas d'échec l'appel renvoie -1 et positionne errno à :

- *INVALID_SOCKET*.
- *ENOBUFS* : pas assez de mémoire
- *EPROTOTYPE* :protocole non supporté

Fermeture d'une socket

A la fin de l'utilisation, une socket doit être fermée par l'appel :

```
close(sock);
int sock;
```

En cas d'erreur errno vaut *SOCKET_ERR*.

Dans le cas d'utilisation d'une communication en mode connecté, le système va continuer à écrire des données sur la socket. Il est donc préférable d'utiliser l'appel :

```
shutdown(sock, how);
int sock, how;
```

L'argument how vaut :

- 0 : plus de lecture
- 1 : plus d'écriture
- 2 : plus de lecture/écriture

Le domaine AF_UNIX

Les sockets de type AF_UNIX sont utilisées dans le cadre d'une communication entre deux processus tournant sur la même machine.

La communication se fait via un fichier de type socket : adressage par un chemin d'accès (pathname) qui est soit *relatif* ou *absolu* : les chemins relatifs sont à éviter.

Il y a donc création d'un noeud dans le système de fichiers désignant le socket (vnode(), unlink()).

Les processus utilisateurs s'y réfèrent en utilisant son pathname.

Structure d'une socket en AF_UNIX : #include <*sys/un.h*>

```
struct sockaddr_un {
    short    sun_family;
    char     sun_path[108];}
```

Le domaine AF_INET

Les sockets de type AF_INET sont une implémentation des protocoles DARPA Internet :

- ➢ adressage IP
- ➢ protocole TCP ou UDP.

Les sockets AF_INET sont utilisées pour une communication entre deux processus tournant sur des machines différentes mais également sur la même machine (adresse de bouclage).

La communication se fait via un canal réseau de type socket.

Structure d'une socket en AF_INET : #include <*netinet/in.h*>

```
struct sockaddr_in {
    short           sin_family;
    u_short         sin_port;
    struct in_addr  sin_addr;
    char            sin_zero[8];}
```

sockaddr_in et sockaddr_un sont une spécialisation de sockaddr

Association d'un adressage à une socket

Une socket créée doit être adressée avant toute utilisation par l'appel système :

```
bind(sock, addr, addrlen);
int sock, addrlen;
struct sockaddr *addr;
```

Cet appel en cas d'échec renvoie -1 et errno vaut *SOCKET_ERROR*

En AF_UNIX, *addr* indique le chemin d'accès au fichier socket.

En AF_INET, la donnée *addr* n'est pas obligatoire (utilisation de l'interface réseau par défaut). Pour une machine ayant plusieurs interfaces, *addr* vaut :

- soit l'adresse de l'interface utilisée,
- soit *INADDR_ANY* pour toutes les interfaces.

Si dans l'adressage, on utilise 0 comme valeur de port, alors le système attribuera un numéro de port libre (cf. *getsockname()*).

DATAGRAM : caractéristiques

Ce mode ne nécessite pas de connexion.

L'échange se fait par l'envoi d'un message entier, appelé datagram.

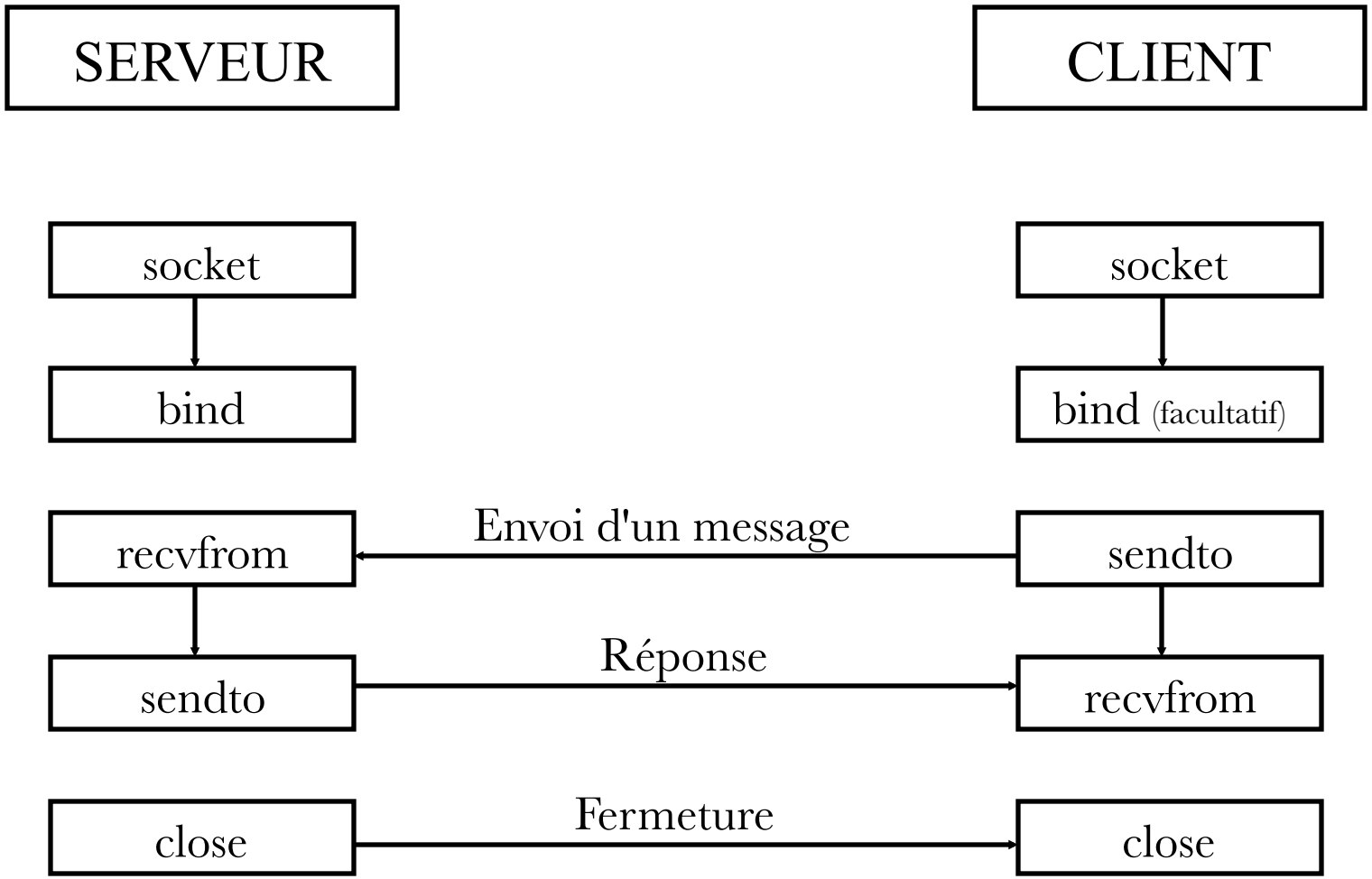
Aucune gestion des erreurs de transmission n'est effectuée.

Le seul traitement d'erreurs effectué est basé sur un checksum de l'entête UDP et d'un TimeOut. L'émetteur doit recevoir un ACK du destinataire durant une fenêtre temporelle définie par le chien de garde.

Ce mode est rapide.

Ce mode de communication nécessite le protocole UDP.

DATAGRAM : modèle de communication



STREAM : caractéristiques

Ce mode nécessite l'établissement d'une connexion.

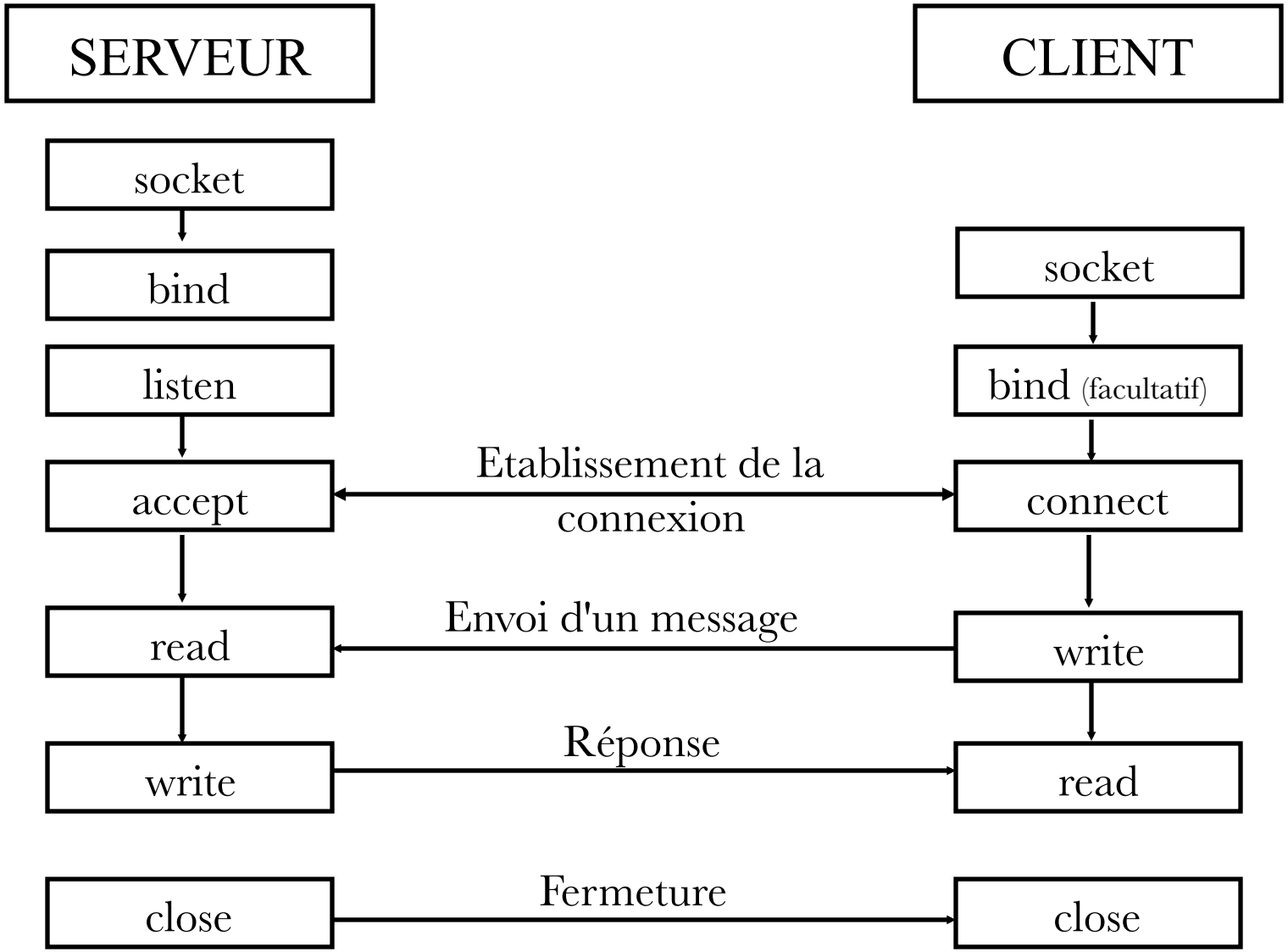
Ce mode offre un transfert de données fiable :

- ➢ **garantie d'acheminement des données,**
- ➢ **respect de l'ordre des paquets : reconstitution du message entier avant délivrance,**
- ➢ **envoi des données urgentes.**

Une gestion automatique des erreurs de transmission (error-free) est assurée par le mode STREAM.

Ce mode nécessite un protocole se caractérisant par une transmission de messages segmentés et d'une gestion des erreurs : TCP.

STREAM : modèle de communication



STREAM : établissement d'une connexion

En mode connecté, le serveur doit établir une connexion avant tout échange.

Le serveur doit être en attente de réception de requêtes de connexion de la part des clients.

Demande de connexion par le client :

```
etat = connect(sock, addr, addrlen);
int etat, sock, addrlen;
struct sockaddr *addr;
```

Un serveur multiple doit indiquer le nombre de clients mis en attente de connexion par l'appel :

```
listen(sock, 5);
```

Acceptation d'une requête de connexion par le serveur :

```
newsock = accept(sock, from, fromlen);
int newsock, sock, *fromlen;
struct sockaddr *from;
/*sock est la socket d'écoute et newsock celle de
  la connexion*/
```

Réception de message en mode connecté

La réception des données se fait par l'appel :

```
nbcar = read(sock, buff, bufflen); /*nbcar = nb de carac. reçus */
int sock, bufflen;
char  *buff;
```

L'appel *read* ne peut être utilisé qu'en mode STREAM (connecté) et renvoie le nombre de caractères reçus, ou 0 si le processus distant a fermé la connexion.

L'appel *read* est construit à partir de l'appel :

```
nbcar = recv(sock, buff, bufflen, flag);
int sock, bufflen, flag;          /* flag = 0 dans le cas du read */
char  *buff;
```

L'argument *flag* permet de configurer la connexion :

- *MSG_OOB* : out of band → lecture de messages urgents.
- *MSG_PEEK* : lecture sans retrait du tampon.

Erreurs de lecture :

- *ENOTCONN* : socket non connectée

Envoi de message en mode connecté

L'envoi des données se fait par l'appel :

```
write(socket, buff, bufflen);
int socket, bufflen;
char  *buff;
```

L'appel *write* ne peut être utilisé qu'en mode STREAM (connecté) et renvoie -1 en cas d'erreur.

L'appel *write* est construit à partir de l'appel :

```
send(sock, buff, bufflen, flag);
int sock, bufflen, flag;          /* flag = 0 dans le cas du write */
char  *buff;
```

L'argument *flag* permet de configurer la connexion :

- *MSG_OOB* : out of band → écriture de messages urgents.
- *MSG_DONTROUTE* : message à destination d'un réseau local.

Erreurs d'écriture :

- *ENOTCONN* : socket non connectée
- *EPIPE* : socket cassée

DATAGRAM : Réception/envoi de message

La réception de données sur une socket non connectée se fait par l'appel :

```
nbcar = recvfrom(sock, buff, bufflen, flag, from, fromlen));
int sock, bufflen, flag, fromlen;  char  *buff;
struct sockaddr *from;
```

L'argument *from* sert à mémoriser l'adresse de l'émetteur. Cette information est utile pour renvoyer une réponse à l'émetteur.

L'envoi de données sur une socket non connectée se fait par l'appel :

```
nbcar = sendto(sock, buff, bufflen, flag, to, tolen));
int sock, bufflen, flag, tolen;      char  *buff;
struct sockaddr *to;
```

L'argument *to* sert à mémoriser l'adresse du destinataire. Cette information est utile pour que le datagramme trouve sa destination.

En cas d'échec, ces fonctions renvoient la valeur -1.

Primitives annexes à l'utilisation des sockets

La programmation des applications réseau nécessite le respect de standards.

Les données doivent être converties au format réseau avant leur transmission.

La résolution de noms doit passer par l'utilisation de mécanismes communs : DNS, NIS, base d'adresses locale.

Passage d'une représentation humaine d'une adresse IP (en décimal) à une représentation machine (4 octets).

Les services offerts par une machine sont nommés (cf /etc/services) → utilisation des noms au lieu des numéros de port.

Primitives de conversion de données

Les données avant transmission sur le réseau doivent être converties dans un format standard appelé "network order".

On dispose des primitives suivantes :

- htons(val) : host order to network order short
- htonl(val) : host order to network order long
- ntohs(val) : network order to host order short
- ntohs(val) : network order to host order long

Primitives de résolution de noms de machines

Structure d'un adressage IP :

```
struct hostent {
    char *h_name;
    char        **h_aliases;
    int         h_addrtype;         /* ~ AF_INET */
    int         h_length; / *  4   */
    char        **h_addr_list; }
#define h_addr h_addr_list[0];
```

Nom d'une machine → Adresse IP :

```
struct hostent gethostbyname (char *name);
```

Adresse IP → Nom d'une machine :

```
struct hostent gethostbyaddr (struct in_addr addr);
```

Adresse IP décimale → Adresse IP machine :

```
char *inet_ntoa (struct in_addr addr);
```

Adresse IP machine → Adresse IP décimale :

```
struct in_addr inet_aton (char *addr_mach);
```

Primitives de résolution de noms de services

Un service réside sur un port et possède un protocole.

Structure d'un service IP :

```
struct servent {
    char *s_name;
    char        **s_aliases;
    int         s_port;
    char        *s_proto; }
```

Nom d'un service → Numéro de port :

```
struct servent getservbyname (char *name, *proto);
```

Numéro de port → Nom d'un service :

```
struct servent getservbyport (port; proto);
int port; char *proto;
```

Création d'un processus fils

Un serveur multiple doit détacher un processus fils pour répondre à une requête de connexion cliente.

Le serveur (processus père) exécute alors l'appel système fork().

Cet appel crée un deuxième processus (fils ou clone) à l'identique du père. La seule différence résulte dans le résultat renvoyé par le fork().

Le fork() renvoit le numéro du processus fils dans le cas du père et la valeur 0 dans le cas du fils.

En cas d'erreur, le résultat vaut -1.

Les deux processus continuent leur évolution en parallèle et ce à partir de l'appel du fork().

Attention à la création en boucle : un premier fils créé par une boucle, créera à partir de cet instant autant de fils que son père.